

CS3004

Software Design & Analysis

Complete Study Notes

National University of Computer & Emerging Sciences
Lahore Campus · Spring 2026

Topics Covered

- SDLC & Good Program Characteristics
- Object Associations
- Class / Use-Case / Sequence / State Diagrams
- SOLID Design Principles
- Design Patterns (8 patterns, C++)
- OOP Review & UML
- Multiple Inheritance
- Activity Diagrams
- OO Metrics (CK Suite)
- Refactoring Basics

Contents

1	SDLC and Characteristics of Good Software	3
1.1	Software Development Life Cycle (SDLC)	3
1.1.1	Phases of SDLC	3
1.1.2	Popular SDLC Models	4
1.2	Characteristics of Good Programs	4
2	OOP Review, UML Basics, and Object Associations	6
2.1	OOP Fundamentals	6
2.1.1	Classes and Objects	6
2.1.2	Inheritance and the is-a Rule	7
2.1.3	Polymorphism	9
2.2	Object Associations	9
2.2.1	Types of Associations	10
2.3	Multiple Inheritance and the Diamond Problem	14
3	UML Diagrams	16
3.1	Class Diagrams	16
3.1.1	Class Diagram Notation	16
3.1.2	Class Diagram Example – Online Store	17
3.2	Use Case Diagrams	17
3.2.1	Elements	17
3.3	Sequence Diagrams	18
3.3.1	Key Elements	18
3.3.2	ATM Withdrawal – Sequence Diagram Description	18
3.4	State Diagrams	19
3.4.1	Elements	19
3.4.2	Order Lifecycle – State Diagram	19
3.5	Activity Diagrams	19
3.5.1	Elements	19
4	SOLID Design Principles	21
4.1	S – Single Responsibility Principle (SRP)	21
4.2	O – Open/Closed Principle (OCP)	22
4.3	L – Liskov Substitution Principle (LSP)	24
4.4	I – Interface Segregation Principle (ISP)	25
4.5	D – Dependency Inversion Principle (DIP)	27

5	Object-Oriented Metrics (CK Suite)	30
5.1	CK Metrics Suite	30
5.1.1	WMC – Weighted Methods per Class	30
5.1.2	DIT – Depth of Inheritance Tree	31
5.1.3	NOC – Number of Children	31
5.1.4	CBO – Coupling Between Objects	31
5.1.5	RFC – Response for a Class	32
5.1.6	LCOM – Lack of Cohesion in Methods	33
5.2	Summary Table of CK Metrics	33
6	Design Patterns	35
6.1	Pattern 1: Factory Method (Creational)	35
6.1.1	Structure	35
6.2	Pattern 2: Singleton (Creational)	38
6.3	Pattern 3: Adapter (Structural)	41
6.4	Pattern 4: Composite (Structural)	44
6.5	Pattern 5: Iterator (Behavioral)	47
6.6	Pattern 6: Observer (Behavioral)	50
6.7	Pattern 7: State (Behavioral)	54
6.8	Pattern 8: Template Method (Behavioral)	59
7	Additional Design Principles	64
7.1	DRY – Don’t Repeat Yourself	64
7.2	KISS – Keep It Simple, Stupid	65
7.3	YAGNI – You Ain’t Gonna Need It	65
7.4	Law of Demeter (LoD)	65
7.5	Composition Over Inheritance	66
8	Quick Reference & Exam Tips	68
8.1	Design Pattern Summary Table	68
8.2	SOLID Principles – One-liner Summary	68
8.3	CK Metrics Quick Reference	69
8.4	Common Exam Mistakes to Avoid	69

Chapter 1

SDLC and Characteristics of Good Software

1.1 Software Development Life Cycle (SDLC)

Definition: SDLC

The **Software Development Life Cycle (SDLC)** is a structured process for planning, creating, testing, and delivering software. It defines a series of phases that a project passes through from inception to retirement.

1.1.1 Phases of SDLC

1. **Requirements Gathering / Analysis** – Understand what the client needs. Deliverable: Software Requirements Specification (SRS).
2. **System Design** – Translate requirements into architecture. Deliverable: Design Document (HLD / LLD).
3. **Implementation (Coding)** – Write the actual code in the chosen language/framework.
4. **Testing** – Unit, integration, system, and acceptance testing.
5. **Deployment** – Release to production environment.
6. **Maintenance** – Bug fixes, enhancements, performance tuning.

1.1.2 Popular SDLC Models

Model	Description	Best For
Waterfall	Sequential phases; no going back	Small, well-defined projects
Iterative	Repeat phases in cycles	Evolving requirements
Spiral	Risk-driven iterative	High-risk, large projects
Agile/Scrum	Short sprints, frequent delivery	Modern web/mobile apps
RUP	Use-case driven, architecture-centric	Enterprise systems

1.2 Characteristics of Good Programs

Definition: Software Quality Attributes

Good software is not just functionally correct – it must exhibit several non-functional qualities.

1. **Correctness** – Does exactly what the specification says.
2. **Reliability** – Works consistently under defined conditions.
3. **Robustness** – Handles unexpected inputs/situations gracefully.
4. **Efficiency** – Uses CPU, memory, and I/O resources wisely.
5. **Usability** – Easy to learn and use.
6. **Maintainability** – Easy to fix and modify.
7. **Portability** – Runs on different platforms without major changes.
8. **Reusability** – Components can be used in other systems.
9. **Readability / Understandability** – Code is self-explanatory.

Practice Questions – Easy to Hard

Easy

1. Name any four phases of SDLC.
2. What does “maintainability” mean in software quality?
3. Which SDLC model is strictly sequential?

Medium

4. Compare the Waterfall model with the Agile model. When would you prefer each?

5. A program crashes when the user enters a negative number where a positive is expected. Which quality attribute is violated? Explain.

Hard

6. You are designing a healthcare management system for a hospital. Evaluate which SDLC model you would choose and justify your answer with at least three quality attributes that must be prioritised.
7. Explain how the Spiral model addresses the shortcomings of the Waterfall model with respect to risk management.

Chapter 2

OOP Review, UML Basics, and Object Associations

2.1 OOP Fundamentals

2.1.1 Classes and Objects

Definition: Class and Object

A **class** is a blueprint defining attributes (data) and methods (behaviour). An **object** is a runtime instance of a class with its own state.

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class BankAccount {
6 private:
7     string owner;
8     double balance;
9     int accountNumber;
10
11 public:
12     // Constructor
13     BankAccount(const string& ownerName, int accNum, double
        initialBalance = 0.0)
14         : owner(ownerName), accountNumber(accNum), balance(
            initialBalance) {}
15
16     // Deposit money
17     void deposit(double amount) {
18         if (amount > 0)
19             balance += amount;
20     }
21
22     // Withdraw money
23     bool withdraw(double amount) {
```

```
24     if (amount > 0 && balance >= amount) {
25         balance -= amount;
26         return true;
27     }
28     return false; // Insufficient funds
29 }
30
31 double getBalance() const { return balance; }
32 string getOwner() const { return owner; }
33
34 void display() const {
35     cout << "Account[" << accountNumber << "] Owner: " <<
36         owner
37         << " Balance: Rs." << balance << endl;
38 }
39
40 int main() {
41     BankAccount acc1("Ali", 1001, 5000.0);
42     BankAccount acc2("Sara", 1002);
43
44     acc1.deposit(2000);
45     acc1.withdraw(500);
46     acc1.display(); // Balance: Rs. 6500
47
48     acc2.deposit(10000);
49     acc2.display(); // Balance: Rs. 10000
50     return 0;
51 }
```

Listing 2.1: Class and Object in C++

2.1.2 Inheritance and the is-a Rule

Definition: Inheritance

Inheritance allows a derived (child) class to acquire properties and behaviour of a base (parent) class. The **is-a rule**: use inheritance only when “Child IS-A Parent” makes logical sense. E.g., Dog IS-A Animal – correct. Car IS-A Engine – wrong (use composition instead).

```
1 #include <iostream>
2 #include <cmath>
3 using namespace std;
4
5 // Base class
6 class Shape {
7 protected:
8     string color;
9 public:
```



```
10     Shape(const string& c = "white") : color(c) {}
11     virtual double area() const = 0;           // Pure virtual
12     virtual double perimeter() const = 0;      // Pure virtual
13     virtual void display() const {
14         cout << "Shape [" << color << "]" area=" << area() <<
15         endl;
16     }
17     virtual ~Shape() {}
18 };
19 // Derived class
20 class Circle : public Shape {
21 private:
22     double radius;
23 public:
24     Circle(double r, const string& c = "red") : Shape(c), radius
25         (r) {}
26     double area() const override { return M_PI * radius * radius
27         ; }
28     double perimeter() const override { return 2 * M_PI * radius
29         ; }
30     void display() const override {
31         cout << "Circle [r=" << radius << ", color=" << color
32         << "]" area=" << area() << endl;
33     }
34 };
35
36 class Rectangle : public Shape {
37 private:
38     double width, height;
39 public:
40     Rectangle(double w, double h, const string& c = "blue")
41         : Shape(c), width(w), height(h) {}
42     double area() const override { return width * height; }
43     double perimeter() const override { return 2 * (width +
44         height); }
45 };
46
47 int main() {
48     Shape* shapes[] = { new Circle(5), new Rectangle(4, 6) };
49     for (auto s : shapes) {
50         s->display();
51         delete s;
52     }
53     return 0;
54 }
```

Listing 2.2: Inheritance – Shape Hierarchy

2.1.3 Polymorphism

Definition: Polymorphism

Polymorphism (Greek: “many forms”) allows the same interface to be used with different underlying types.

- **Compile-time (static):** Function overloading, operator overloading.
- **Runtime (dynamic):** Virtual functions, overriding in derived classes.

```
1 class Animal {
2 public:
3     virtual void speak() const { cout << "... " << endl; }
4     virtual ~Animal() {}
5 };
6
7 class Dog : public Animal {
8 public:
9     void speak() const override { cout << "Woof!" << endl; }
10 };
11
12 class Cat : public Animal {
13 public:
14     void speak() const override { cout << "Meow!" << endl; }
15 };
16
17 // Polymorphic call through base pointer
18 void makeSpeak(Animal* a) { a->speak(); }
19
20 int main() {
21     Animal* animals[] = { new Dog(), new Cat(), new Dog() };
22     for (auto a : animals) { makeSpeak(a); delete a; }
23     return 0;
24 }
```

Listing 2.3: Runtime Polymorphism with virtual functions

2.2 Object Associations

2.2.1 Types of Associations

Type	Meaning	Example
Simple Association	“uses” relationship	Student uses Library
Aggregation	“has-a” (weak ownership)	Team has Players (players exist without team)
Composition	“has-a” (strong ownership)	House has Rooms (rooms die with house)
Dependency	“depends on temporarily”	Order depends on Payment-Gateway

Multiplicities

Notation	Meaning
1	Exactly one
0..1	Zero or one (optional)
* or 0..*	Zero or more
1..*	One or more (at least one)
m..n	Between m and n

Aggregation vs Composition in C++

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  using namespace std;
5
6  class Professor {
7      string name;
8  public:
9      Professor(const string& n) : name(n) {}
10     string getName() const { return name; }
11 };
12
13 // AGGREGATION: Department has Professors, but Professors
14 // exist independently (they could belong to multiple depts)
15 class Department {
16     string deptName;
17     vector<Professor*> professors; // Pointer -- aggregation
18 public:
19     Department(const string& n) : deptName(n) {}
20     void addProfessor(Professor* p) { professors.push_back(p); }
21     void display() const {
22         cout << "Dept: " << deptName << endl;
23         for (auto p : professors)
24             cout << "  Prof: " << p->getName() << endl;
25     }
26     // Department does NOT delete professors

```

```

27 };
28
29 int main() {
30     Professor p1("Dr. Ali"), p2("Dr. Sara");
31     Department cs("Computer Science"), math("Mathematics");
32     cs.addProfessor(&p1);
33     cs.addProfessor(&p2);
34     math.addProfessor(&p2); // Dr. Sara is in two depts
35     cs.display();
36     math.display();
37     return 0;
38 }

```

Listing 2.4: Aggregation – Department and Professor

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  using namespace std;
5
6  class Room {
7      string roomType;
8      double area;
9  public:
10     Room(const string& t, double a) : roomType(t), area(a) {}
11     void display() const {
12         cout << "    Room[" << roomType << ", " << area << " sqft]
13             << endl;
14     };
15
16     // COMPOSITION: House owns Rooms. Rooms cannot exist without
17     // House.
18     class House {
19         string address;
20         vector<Room> rooms; // By value -- composition
21     public:
22         House(const string& addr) : address(addr) {}
23         void addRoom(const string& type, double area) {
24             rooms.emplace_back(type, area);
25         }
26         void display() const {
27             cout << "House at: " << address << endl;
28             for (const auto& r : rooms) r.display();
29         }
30         // Rooms are destroyed automatically with House
31     };
32
33     int main() {
34         House h("123 Model Town, Lahore");
35         h.addRoom("Living Room", 400);
36         h.addRoom("Bedroom", 200);

```

```
36     h.addRoom("Kitchen", 150);
37     h.display();
38     return 0;
39 }
```

Listing 2.5: Composition – House and Room

Many-to-Many Association

```
1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <algorithm>
5  using namespace std;
6
7  class Course; // Forward declaration
8
9  class Student {
10     string name;
11     int id;
12     vector<Course*> enrolledCourses;
13 public:
14     Student(const string& n, int i) : name(n), id(i) {}
15     void enroll(Course* c);
16     string getName() const { return name; }
17     void showCourses() const;
18 };
19
20 class Course {
21     string title;
22     string code;
23     vector<Student*> students;
24 public:
25     Course(const string& c, const string& t) : code(c), title(t) {}
26     void addStudent(Student* s) { students.push_back(s); }
27     string getCode() const { return code; }
28     string getTitle() const { return title; }
29     void showStudents() const {
30         cout << "Course " << code << ": " << title << endl;
31         for (auto s : students)
32             cout << "    Student: " << s->getName() << endl;
33     }
34 };
35
36 void Student::enroll(Course* c) {
37     enrolledCourses.push_back(c);
38     c->addStudent(this);
39 }
40
41 void Student::showCourses() const {
```

```

42     cout << "Student: " << name << " enrolled in:" << endl;
43     for (auto c : enrolledCourses)
44         cout << " " << c->getCode() << " - " << c->getTitle()
45             << endl;
46 }
47 int main() {
48     Student s1("Ali", 101), s2("Sara", 102), s3("Ahmed", 103);
49     Course c1("CS101", "Intro to CS"), c2("CS201", "Data
50         Structures");
51     s1.enroll(&c1); s1.enroll(&c2);
52     s2.enroll(&c1);
53     s3.enroll(&c2);
54     s1.showCourses();
55     c1.showStudents();
56     return 0;
57 }

```

Listing 2.6: Many-to-Many – Student and Course

Reflexive Association

```

1  class Employee {
2      string name;
3      Employee* manager;           // Reflexive: Employee manages
                                   // Employee
4      vector<Employee*> reports;    // Subordinates
5  public:
6      Employee(const string& n) : name(n), manager(nullptr) {}
7      void setManager(Employee* mgr) {
8          manager = mgr;
9          mgr->reports.push_back(this);
10     }
11     void display() const {
12         cout << name;
13         if (manager) cout << " (reports to: " << manager->name
14             << ")";
15         cout << endl;
16     }
17 };

```

Listing 2.7: Reflexive Association – Employee manages Employees

2.3 Multiple Inheritance and the Diamond Problem

Definition: Diamond Problem

When a class D inherits from both B and C, and both inherit from A, there are two copies of A in D. This ambiguity is the **diamond problem**.

```
1 #include <iostream>
2 using namespace std;
3
4 class Vehicle {
5 public:
6     int speed;
7     Vehicle(int s = 0) : speed(s) {}
8     void showSpeed() { cout << "Speed: " << speed << endl; }
9 };
10
11 // Without virtual: TWO copies of Vehicle in AmphibiousVehicle
12 class Car : virtual public Vehicle {      // virtual keyword
13     // solves diamond
14     public: Car(int s) : Vehicle(s) {}
15 };
16 class Boat : virtual public Vehicle {
17     public: Boat(int s) : Vehicle(s) {}
18 };
19
20 // Diamond: AmphibiousVehicle -> Car -> Vehicle
21 //                               -> Boat -> Vehicle
22 class AmphibiousVehicle : public Car, public Boat {
23 public:
24     // With virtual inheritance, only ONE Vehicle sub-object
25     AmphibiousVehicle(int s) : Vehicle(s), Car(s), Boat(s) {}
26 };
27
28 int main() {
29     AmphibiousVehicle av(120);
30     av.showSpeed(); // No ambiguity because of virtual
31                     // inheritance
32     return 0;
33 }
```

Listing 2.8: Diamond Problem and Virtual Inheritance in C++

Key Insight

In C++, use `virtual public Base` to solve the diamond problem. Java avoids it entirely by not allowing multiple class inheritance – only interface inheritance is permitted.

Practice Questions – Easy to Hard**Easy**

1. What is the “is-a” rule? Give two valid and two invalid examples.
2. Distinguish between aggregation and composition with a real-world example.
3. What is the difference between function overloading and function overriding?

Medium

4. Implement a one-to-many association between **Library** and **Book** in C++. A library can have many books. Show how to add, remove, and list books.
5. Explain why Java does not support multiple inheritance of classes. How does it allow reuse via interfaces instead?

Hard

6. Design a University system with classes: **University**, **Department**, **Professor**, **Course**, **Student**. Identify all associations (type, multiplicity) and implement in C++.
7. What is the diamond problem? Demonstrate it in C++ without virtual inheritance (showing the error/ambiguity), then fix it with virtual inheritance.

Chapter 3

UML Diagrams

3.1 Class Diagrams

Definition: Class Diagram

A **class diagram** is a static structural diagram showing the system's classes, their attributes, methods, and relationships (inheritance, association, aggregation, composition, dependency).

3.1.1 Class Diagram Notation

Symbol / Notation	Meaning
+	Public visibility
-	Private visibility
#	Protected visibility
~	Package visibility
Solid line + open arrow	Association
Hollow diamond + line	Aggregation
Filled diamond + line	Composition
Hollow triangle arrow	Inheritance (Generalisation)
Dashed line + arrow	Dependency
Dashed line + hollow triangle	Interface realisation

3.1.2 Class Diagram Example – Online Store

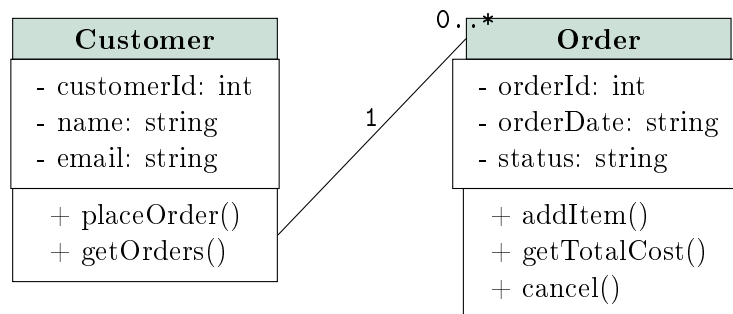


Figure 3.1: Partial Class Diagram for Online Store (conceptual)

3.2 Use Case Diagrams

Definition: Use Case Diagram

A **use case diagram** shows the interactions between **actors** (users or external systems) and the system. It captures **functional requirements** at a high level.

3.2.1 Elements

- **Actor:** Stick figure – represents a role (user, admin, external system).
- **Use Case:** Oval – represents a system function/feature.
- **System Boundary:** Rectangle enclosing all use cases.
- **Association:** Line between actor and use case.
- **«include»:** One use case always calls another.
- **«extend»:** One use case optionally extends another.
- **Generalisation:** Actor or use case inheritance.

Use Case Description Template

Field	Description
Use Case Name	Login
Actor	Registered User
Precondition	User has a valid account
Main Flow	1. User opens login page 2. Enters username/password 3. System validates credentials 4. System shows dashboard
Alternate Flow	3a. Invalid credentials → show error, allow retry (max 3 times)
Postcondition	User is authenticated and redirected to home

3.3 Sequence Diagrams

Definition: Sequence Diagram

A **sequence diagram** is a dynamic/interaction diagram that shows how objects communicate over **time**. It shows the **order of messages** exchanged between lifelines.

3.3.1 Key Elements

- **Lifeline**: Vertical dashed line below an object/actor box.
- **Activation bar**: Thin rectangle on lifeline showing object is active.
- **Synchronous message**: Solid line with filled arrow ($\rightarrow$) – caller waits.
- **Asynchronous message**: Solid line with open arrow – caller does not wait.
- **Return message**: Dashed line with open arrow ($--\rightarrow$).
- **Self-message**: Arrow looping back to same lifeline.
- **Create**: Dashed arrow to a new lifeline.
- **Destroy**: X at end of lifeline.
- **Combined Fragments**: alt (if-else), opt (optional), loop, par (parallel).

3.3.2 ATM Withdrawal – Sequence Diagram Description

1. Customer inserts card $\rightarrow$ ATM reads card.
2. ATM $\rightarrow$ Bank: `validateCard(cardNumber)`
3. Bank $\rightarrow$ ATM: `cardValid = true`
4. ATM prompts for PIN; Customer enters PIN.
5. ATM $\rightarrow$ Bank: `validatePIN(pin)`
6. Bank $\rightarrow$ ATM: `pinValid = true`
7. Customer selects “Withdraw”, enters amount.
8. ATM $\rightarrow$ Bank: `debitAccount(accountId, amount)`
9. Bank checks balance (self-message).
10. Bank $\rightarrow$ ATM: `transactionApproved`
11. ATM dispenses cash, prints receipt.

3.4 State Diagrams

Definition: State Diagram (State Machine)

A **state diagram** models the **dynamic behaviour** of a single object by showing all possible states it can be in, and the **transitions** triggered by events.

3.4.1 Elements

- **State**: Rounded rectangle.
- **Initial state**: Filled black circle.
- **Final state**: Filled black circle with outer ring.
- **Transition**: Arrow with label: `event [guard] / action`.
- **Entry action**: Executed on entering a state (`entry /`).
- **Exit action**: Executed on leaving a state (`exit /`).
- **Do action**: Executed while in a state (`do /`).

3.4.2 Order Lifecycle – State Diagram

States: **New** $\xrightarrow{\text{submit}}$ **Pending** $\xrightarrow{\text{pay}}$ **Confirmed** $\xrightarrow{\text{ship}}$ **Shipped** $\xrightarrow{\text{deliver}}$ **Delivered**
Cancel transitions possible from New, Pending, Confirmed $\rightarrow$ **Cancelled**.

3.5 Activity Diagrams

Definition: Activity Diagram

An **activity diagram** models the **flow of control** (or data) through a system. It is similar to a flowchart but also models parallel activities using **forks** and **joins** (synchronisation bars).

3.5.1 Elements

- **Activity (Action)**: Rounded rectangle.
- **Decision**: Diamond with multiple outgoing guarded transitions.
- **Fork**: Thick horizontal bar with one input, multiple outputs (parallel start).
- **Join**: Thick horizontal bar with multiple inputs, one output (synchronisation).
- **Swimlane**: Partition showing which actor/component performs each activity.

Practice Questions – Easy to Hard – UML**Easy**

1. What is the difference between a class diagram and an object diagram?
2. What does `«include»` mean in a use case diagram?
3. List three elements of a sequence diagram.

Medium

4. Draw (describe) a use case diagram for an ATM system with actors: Customer, Bank. Identify at least 5 use cases.
5. Draw a state diagram for a traffic light system with states: Red, Green, Yellow.
6. Explain the difference between `«include»` and `«extend»` in use cases with examples.

Hard

7. For an online food delivery system, design: (a) a class diagram with at least 5 classes, (b) a use case diagram, and (c) a sequence diagram for the “Place Order” use case.
8. Design an activity diagram with swimlanes for a hotel booking process involving: Customer, Web System, Payment Gateway, Hotel Manager.

Chapter 4

SOLID Design Principles

Definition: SOLID

SOLID is an acronym for five object-oriented design principles introduced by Robert C. Martin. They guide writing maintainable, extensible, and testable code.

Letter	Principle
S	Single Responsibility Principle (SRP)
O	Open/Closed Principle (OCP)
L	Liskov Substitution Principle (LSP)
I	Interface Segregation Principle (ISP)
D	Dependency Inversion Principle (DIP)

4.1 S – Single Responsibility Principle (SRP)

Definition: SRP

“A class should have only one reason to change.” – Robert C. Martin.
Each class should have a single, well-defined responsibility.

```
1 // BAD: This class does too much -- violates SRP
2 class Employee {
3 public:
4     string name;
5     double salary;
6     void calculatePay() { /* payroll logic */ }
7     void saveToDatabase() { /* DB persistence */ }
8     void generateReport() { /* report formatting */ }
9     // Three reasons to change: payroll, DB schema, report
10    format
};
```

Listing 4.1: SRP Violation

```
1 // GOOD: Each class has ONE responsibility
2 class Employee {
```

```

3 public:
4     string name;
5     double salary;
6     Employee(const string& n, double s) : name(n), salary(s) {}
7 };
8
9 class PayrollCalculator {
10 public:
11     double calculate(const Employee& e) {
12         return e.salary * 1.1; // Example: 10% bonus
13     }
14 };
15
16 class EmployeeRepository {
17 public:
18     void save(const Employee& e) {
19         // DB logic: INSERT INTO employees ...
20         cout << "Saving " << e.name << " to DB" << endl;
21     }
22     Employee load(int id) {
23         // DB logic: SELECT ...
24         return Employee("Loaded", 0);
25     }
26 };
27
28 class EmployeeReportGenerator {
29 public:
30     void generateReport(const Employee& e) {
31         cout << "=== Employee Report ===" << endl;
32         cout << "Name: " << e.name << endl;
33         cout << "Salary: " << e.salary << endl;
34     }
35 };

```

Listing 4.2: SRP Correct – Separate responsibilities

4.2 O – Open/Closed Principle (OCP)

Definition: OCP

“Software entities should be open for extension, but closed for modification.”
 You should be able to add new behaviour without changing existing code.

```

1 // BAD: Every new shape requires modifying existing class
2 class AreaCalculator {
3 public:
4     double calculate(const string& shapeType, double dim1,
5                     double dim2 = 0) {
6         if (shapeType == "circle") return 3.14159 * dim1 * dim1;
7         else if (shapeType == "rect") return dim1 * dim2;
8     }
9 };

```

```

7         // Must modify this method for every new shape!
8         return 0;
9     }
10 };

```

Listing 4.3: OCP Violation – if/else for each shape

```

1  #include <iostream>
2  #include <vector>
3  #include <memory>
4  #include <cmath>
5  using namespace std;
6
7  // Abstract base -- closed for modification
8  class Shape {
9  public:
10     virtual double area() const = 0;
11     virtual ~Shape() = default;
12 };
13
14 // Open for extension -- add new shapes without changing
15 // AreaCalculator
16 class Circle : public Shape {
17     double r;
18 public:
19     Circle(double radius) : r(radius) {}
20     double area() const override { return M_PI * r * r; }
21 };
22
23 class Rectangle : public Shape {
24     double w, h;
25 public:
26     Rectangle(double width, double height) : w(width), h(height) {}
27     double area() const override { return w * h; }
28 };
29
30 class Triangle : public Shape {
31     double base, height;
32 public:
33     Triangle(double b, double h) : base(b), height(h) {}
34     double area() const override { return 0.5 * base * height; }
35 };
36
37 // AreaCalculator never needs to change
38 class AreaCalculator {
39 public:
40     double totalArea(const vector<shared_ptr<Shape>>& shapes)
41         const {
42         double total = 0;
43         for (const auto& s : shapes) total += s->area();
44         return total;
45     }
46 };

```



```

43     }
44 };
45
46 int main() {
47     vector<shared_ptr<Shape>> shapes = {
48         make_shared<Circle>(5),
49         make_shared<Rectangle>(4, 6),
50         make_shared<Triangle>(3, 8)
51     };
52     AreaCalculator calc;
53     cout << "Total area: " << calc.totalArea(shapes) << endl;
54     return 0;
55 }

```

Listing 4.4: OCP Correct – Extend via polymorphism

4.3 L – Liskov Substitution Principle (LSP)

Definition: LSP

“Objects of a derived class must be substitutable for objects of the base class without altering correctness of the program.” – Barbara Liskov, 1987.

If D is a subtype of B, then wherever B is used, D can replace it and the program still works correctly.

```

1  // BAD: Square IS-A Rectangle mathematically, but violates LSP
2  class Rectangle {
3  protected:
4      double width, height;
5  public:
6      Rectangle(double w, double h) : width(w), height(h) {}
7      virtual void setWidth(double w) { width = w; }
8      virtual void setHeight(double h) { height = h; }
9      double area() const { return width * height; }
10 };
11
12 class Square : public Rectangle {
13 public:
14     Square(double side) : Rectangle(side, side) {}
15     void setWidth(double w) override { width = height = w; } //
16         Both change!
17     void setHeight(double h) override { width = height = h; }
18 };
19
20 // This test function expects Rectangle behaviour
21 void testArea(Rectangle& r) {
22     r.setWidth(4);
23     r.setHeight(5);
24     // Expected: 4*5 = 20
25     // For Square: 5*5 = 25 -- WRONG! LSP violated

```

```

25     cout << "Area: " << r.area() << endl;
26 }
27
28 int main() {
29     Rectangle rect(2, 3);
30     testArea(rect); // Prints 20 -- correct
31
32     Square sq(3);
33     testArea(sq);   // Prints 25 -- WRONG, LSP violated
34     return 0;
35 }

```

Listing 4.5: LSP Violation – Square/Rectangle paradox

```

1 // GOOD: Both Square and Rectangle implement Shape independently
2 class Shape {
3 public:
4     virtual double area() const = 0;
5     virtual ~Shape() = default;
6 };
7
8 class Rectangle : public Shape {
9     double w, h;
10 public:
11     Rectangle(double w, double h) : w(w), h(h) {}
12     void setWidth(double nw) { w = nw; }
13     void setHeight(double nh) { h = nh; }
14     double area() const override { return w * h; }
15 };
16
17 class Square : public Shape {
18     double side;
19 public:
20     Square(double s) : side(s) {}
21     void setSide(double s) { side = s; }
22     double area() const override { return side * side; }
23 };
24 // No dangerous substitution -- both are independently correct
    Shapes

```

Listing 4.6: LSP Fix – Common abstract base

4.4 I – Interface Segregation Principle (ISP)

Definition: ISP

“Clients should not be forced to depend on interfaces they do not use.”
 Split large interfaces into smaller, more specific ones.

```

1 // BAD: One fat interface forces Printer and Scanner to
  // implement everything
2 class IDevice {
3 public:
4     virtual void print(const string& doc) = 0;
5     virtual void scan(const string& doc) = 0;
6     virtual void fax(const string& doc) = 0;
7     virtual void staple() = 0;
8 };
9
10 // SimplePrinter can only print, but forced to implement scan/
  // fax/staple
11 class SimplePrinter : public IDevice {
12 public:
13     void print(const string& doc) override { cout << "Printing:
14         " << doc << endl; }
15     void scan(const string& doc) override { throw runtime_error(
16         "Not supported"); }
17     void fax(const string& doc) override { throw runtime_error(
18         "Not supported"); }
19     void staple() override { throw runtime_error("
20         Not supported"); }
21 };

```

Listing 4.7: ISP Violation – Fat interface

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class IPrintable {
6 public:
7     virtual void print(const string& doc) = 0;
8     virtual ~IPrintable() = default;
9 };
10
11 class IScannable {
12 public:
13     virtual void scan(const string& doc) = 0;
14     virtual ~IScannable() = default;
15 };
16
17 class IFaxable {
18 public:
19     virtual void fax(const string& doc) = 0;
20     virtual ~IFaxable() = default;
21 };
22
23 // SimplePrinter only implements what it can do
24 class SimplePrinter : public IPrintable {
25 public:
26     void print(const string& doc) override {

```

```

27         cout << "SimplePrinter: Printing '" << doc << "'" <<
           endl;
28     }
29 };
30
31 // AllInOnePrinter implements multiple small interfaces
32 class AllInOnePrinter : public IPrintable, public IScannable,
   public IFaxable {
33 public:
34     void print(const string& doc) override { cout << "AIO: Print
           " << doc << endl; }
35     void scan(const string& doc) override { cout << "AIO: Scan
           " << doc << endl; }
36     void fax(const string& doc) override { cout << "AIO: Fax "
           << doc << endl; }
37 };
38
39 int main() {
40     SimplePrinter sp;
41     sp.print("Report.pdf");
42
43     AllInOnePrinter aio;
44     aio.print("Report.pdf");
45     aio.scan("Contract.pdf");
46     aio.fax("Invoice.pdf");
47     return 0;
48 }

```

Listing 4.8: ISP Correct – Segregated interfaces

4.5 D – Dependency Inversion Principle (DIP)

Definition: DIP

“High-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions.”

```

1 // BAD: OrderProcessor (high-level) directly depends on
   MySQLDatabase (low-level)
2 class MySQLDatabase {
3 public:
4     void saveOrder(int orderId) {
5         cout << "Saving order " << orderId << " to MySQL" <<
           endl;
6     }
7 };
8
9 class OrderProcessor {

```

```

10     MySQLDatabase db; // Tight coupling! Can't switch to MongoDB
        without rewrite
11 public:
12     void processOrder(int orderId) {
13         // ... business logic ...
14         db.saveOrder(orderId);
15     }
16 };

```

Listing 4.9: DIP Violation – High-level depends on low-level

```

1 #include <iostream>
2 #include <memory>
3 using namespace std;
4
5 // Abstraction (interface)
6 class IDatabase {
7 public:
8     virtual void saveOrder(int orderId) = 0;
9     virtual ~IDatabase() = default;
10 };
11
12 // Low-level detail 1
13 class MySQLDatabase : public IDatabase {
14 public:
15     void saveOrder(int orderId) override {
16         cout << "MySQL: Saving order " << orderId << endl;
17     }
18 };
19
20 // Low-level detail 2
21 class MongoDBDatabase : public IDatabase {
22 public:
23     void saveOrder(int orderId) override {
24         cout << "MongoDB: Saving order " << orderId << endl;
25     }
26 };
27
28 // High-level module -- depends on abstraction, not concrete
    class
29 class OrderProcessor {
30     shared_ptr<IDatabase> db; // Depends on IDatabase, not
        concrete type
31 public:
32     // Dependency injected via constructor
33     OrderProcessor(shared_ptr<IDatabase> database) : db(database) {}
34     void processOrder(int orderId) {
35         cout << "Processing order " << orderId << endl;
36         db->saveOrder(orderId);
37     }
38 };

```

```
39
40 int main() {
41     // Swap implementations without changing OrderProcessor
42     auto mysqlDB = make_shared<MySQLDatabase>();
43     OrderProcessor p1(mysqlDB);
44     p1.processOrder(1001);
45
46     auto mongoDB = make_shared<MongoDatabase>();
47     OrderProcessor p2(mongoDB);
48     p2.processOrder(1002);
49     return 0;
50 }
```

Listing 4.10: DIP Correct – Both depend on abstraction

Practice Questions – Easy to Hard – SOLID

Easy

1. What does SRP stand for? Give a real-world class that violates it and explain why.
2. State the Open/Closed Principle in your own words.
3. What is the key idea behind the Dependency Inversion Principle?

Medium

4. Demonstrate LSP violation with the Square-Rectangle problem and provide a correct redesign.
5. A class `Worker` has methods: `work()`, `eat()`, `sleep()`, `maintainMachinery()`. Which SOLID principle does this violate? Redesign it.
6. Show how ISP is violated in a media player interface with methods: `playAudio()`, `playVideo()`, `playDVD()`, `streamOnline()`. Provide the correct design.

Hard

7. Design a notification system (email, SMS, push) that adheres to all five SOLID principles. Show the C++ class hierarchy and explain which principle each design decision addresses.
8. Refactor the following class to comply with all SOLID principles: a `ReportManager` class that reads data from a CSV, formats it as HTML, and emails the report.

Chapter 5

Object-Oriented Metrics (CK Suite)

Definition: OO Metrics

Object-Oriented Metrics provide quantitative measures to assess software quality in OO systems. The **CK Metrics Suite** (Chidamber & Kemerer, 1994) is the most widely used set of OO metrics.

5.1 CK Metrics Suite

5.1.1 WMC – Weighted Methods per Class

Definition: WMC

$WMC = \sum_{i=1}^n c_i$ where c_i is the complexity of method i (often cyclomatic complexity). If all weights = 1, WMC = number of methods.

Interpretation: High $WMC \Rightarrow$ class is too complex, hard to maintain, likely violates SRP.

Example: WMC Calculation

```
1 class OrderService {
2     void createOrder() { /* complexity 3 */ }
3     void cancelOrder() { /* complexity 2 */ }
4     void updateOrder() { /* complexity 4 */ }
5     void listOrders() { /* complexity 1 */ }
6 };
7 // WMC = 3 + 2 + 4 + 1 = 10
8 // If we just count methods: WMC = 4
```

Guideline: $WMC \leq 20$ (Lorenz & Kidd). Higher values suggest refactoring.

5.1.2 DIT – Depth of Inheritance Tree

Definition: DIT

DIT = length of the longest path from a class to the root of the inheritance tree.

Interpretation:

- *High DIT* (> 5): Complex, hard to understand, many inherited methods.
- *Low DIT* ($= 0$): No inheritance, limited reuse potential.

Recommended: DIT between 2 and 5.

Example: DIT Example

```
Vehicle (DIT=0)
  --> Car (DIT=1)
    --> ElectricCar (DIT=2)
      --> TeslaModelS (DIT=3)

TeslaModelS.DIT = 3
```

5.1.3 NOC – Number of Children

Definition: NOC

NOC = number of immediate subclasses of a class.

Interpretation:

- High NOC: High reuse, but if the base class changes, many children are affected.
- Too many children may indicate improper abstraction or missing intermediate classes.

5.1.4 CBO – Coupling Between Objects

Definition: CBO

CBO = number of classes to which a given class is coupled (via method calls, variable usage, inheritance, etc.).

Interpretation:

- High CBO $\Rightarrow$ low modularity, hard to test in isolation, ripple effects on change.
- Aim for *loose coupling* (low CBO) and *high cohesion*.

Guideline: $CBO \leq 14$ (Harrison et al.)

```
1 class OrderService {
```



```

2      CustomerRepository custRepo; // Coupled to
      CustomerRepository
3      ProductCatalog catalog;      // Coupled to ProductCatalog
4      PaymentGateway payment;      // Coupled to PaymentGateway
5      EmailService email;          // Coupled to EmailService
6      // CBO = 4
7      void placeOrder() {
8          auto c = custRepo.find(1);
9          auto p = catalog.getProduct(101);
10         payment.charge(c, p.price);
11         email.sendConfirmation(c);
12     }
13 };

```

Listing 5.1: CBO Example

5.1.5 RFC – Response for a Class

Definition: RFC

$RFC = |M| + |R|$ where M is the set of methods of the class, and R is the set of methods called by those methods (from other classes).

Interpretation: High RFC $\Rightarrow$ complex testing, deeper call chains, harder debugging.

Example: RFC Calculation

```

1  class Booking {
2      // Own methods: makeBooking(), cancelBooking(),
      getDetails()    |M| = 3
3      void makeBooking() {
4          hotel.checkAvailability();    // Called method
5          payment.processPayment();    // Called method
6          email.sendConfirmation();    // Called method
7      }
8      void cancelBooking() {
9          payment.refund();              // Called method
10         hotel.releaseRoom();          // Called method
11     }
12     // |R| = 5 unique called methods
13     // RFC = 3 + 5 = 8
14 };

```

5.1.6 LCOM – Lack of Cohesion in Methods

Definition: LCOM (LCOM1)

Given a class with n methods $M_1, M_2, \dots, M_n$ and instance variables $V_1, \dots, V_k$:

Let I_j = set of instance variables used by method M_j .

$P = \{(M_i, M_j) \mid I_i \cap I_j = \emptyset\}$ (pairs with no shared variables)

$Q = \{(M_i, M_j) \mid I_i \cap I_j \neq \emptyset\}$ (pairs sharing at least one variable)

$LCOM = \max(|P| - |Q|, 0)$

Interpretation: $LCOM = 0$ (perfectly cohesive). High LCOM $\Rightarrow$ class should be split.

Example: LCOM Calculation

```

1  class PersonBad {
2      string name;    // V1
3      int age;        // V2
4      double salary; // V3
5
6      string getName() { return name; }    // Uses {V1}
7      int getAge() { return age; }         // Uses {V2}
8      double getSalary() { return salary; } // Uses {V3}
9      // Methods share NO variables:
10     // P = {(M1,M2), (M1,M3), (M2,M3)} => |P| = 3
11     // Q = {} => |Q| = 0
12     // LCOM = 3 - 0 = 3 (LOW cohesion -- consider splitting)
13 };

```

5.2 Summary Table of CK Metrics

Metric	Full Name	Threshold	High Value Indicates
WMC	Weighted Methods per Class	≤ 20	Complex class, split needed
DIT	Depth of Inheritance Tree	2–5	Deep hierarchies, brittle
NOC	Number of Children	context-dependent	High reuse or misuse
CBO	Coupling Between Objects	≤ 14	High coupling, poor modularity
RFC	Response for a Class	≤ 50	Complex testing required
LCOM	Lack of Cohesion in Methods	$= 0$ ideal	Low cohesion, split class

Practice Questions – Easy to Hard – CK Metrics**Easy**

1. What does CBO measure? Why is high CBO bad?
2. If a class has 5 methods each with cyclomatic complexity 2, what is the WMC?
3. What does a LCOM value of 0 indicate?

Medium

4. Calculate DIT for each class: `Object` → `Animal` → `Mammal` → `Dog` → `Labrador`.
5. A class `UserManager` has methods that call 12 other classes' methods and has 8 of its own methods. Calculate RFC. Is this acceptable?
6. Explain the relationship between high CBO and the Dependency Inversion Principle.

Hard

7. Design two versions of a `ReportGenerator` class – one with high LCOM and one with low LCOM. Calculate LCOM for both and suggest refactoring.
8. Given the following class, calculate WMC, CBO, and RFC. Then propose concrete refactorings to improve all three metrics.

```
class SalesManager {
    CustomerDB cdb; InventoryDB idb; EmailService es;
    ReportGenerator rg; Logger logger;
    void processSale(int custId, int itemId, int qty) {
        /* complexity 5 */ }
    void generateReport(int month) {
        /* complexity 3 */ }
    void notifyCustomer(int custId) {
        /* complexity 2 */ }
    void updateInventory(int itemId, int qty) {
        /* complexity 4 */ }
};
```

Chapter 6

Design Patterns

Definition: Design Pattern

A **design pattern** is a reusable, proven solution to a commonly occurring problem in software design. Introduced by the **Gang of Four (GoF)** – Gamma, Helm, Johnson, Vlissides (1994).

Patterns are classified into three categories:

- **Creational:** How objects are created. (Singleton, Factory Method, Abstract Factory, Builder, Prototype)
- **Structural:** How objects are composed. (Adapter, Composite, Decorator, Facade, Flyweight, Proxy, Bridge)
- **Behavioral:** How objects communicate. (Observer, State, Strategy, Template Method, Iterator, Command, ...)

6.1 Pattern 1: Factory Method (Creational)

Definition: Factory Method Pattern

Intent: Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.

Also known as: Virtual Constructor.

6.1.1 Structure

- **Product:** Abstract interface for objects the factory creates.
- **ConcreteProduct:** Implements Product.
- **Creator:** Declares factory method, may implement default logic.
- **ConcreteCreator:** Overrides factory method to return a ConcreteProduct.

```
1 #include <iostream>
2 #include <memory>
3 #include <string>
4 using namespace std;
5
6 // Product Interface
7 class Document {
8 public:
9     virtual void open() = 0;
10    virtual void save() = 0;
11    virtual void close() = 0;
12    virtual ~Document() = default;
13 };
14
15 // Concrete Products
16 class WordDocument : public Document {
17 public:
18     void open() override { cout << "[Word] Opening .docx" <<
19                             endl; }
19     void save() override { cout << "[Word] Saving .docx" <<
20                             endl; }
20     void close() override { cout << "[Word] Closing" <<
21                             endl; }
21 };
22
23 class PDFDocument : public Document {
24 public:
25     void open() override { cout << "[PDF] Opening .pdf" <<
26                             endl; }
26     void save() override { cout << "[PDF] Saving .pdf" <<
27                             endl; }
27     void close() override { cout << "[PDF] Closing" <<
28                             endl; }
28 };
29
30 class SpreadsheetDocument : public Document {
31 public:
32     void open() override { cout << "[Excel] Opening .xlsx" <<
33                             endl; }
33     void save() override { cout << "[Excel] Saving .xlsx" <<
34                             endl; }
34     void close() override { cout << "[Excel] Closing" <<
35                             endl; }
35 };
36
37 // Creator (Abstract)
38 class Application {
39 public:
40     // The Factory Method -- subclasses decide what to create
41     virtual unique_ptr<Document> createDocument() = 0;
42     virtual ~Application() = default;
```

```
43
44     // Template method using factory method
45     void newDocument() {
46         auto doc = createDocument();
47         doc->open();
48         doc->save();
49         doc->close();
50     }
51 };
52
53 // Concrete Creators
54 class WordApplication : public Application {
55 public:
56     unique_ptr<Document> createDocument() override {
57         return make_unique<WordDocument>();
58     }
59 };
60
61 class PDFApplication : public Application {
62 public:
63     unique_ptr<Document> createDocument() override {
64         return make_unique<PDFDocument>();
65     }
66 };
67
68 class SpreadsheetApplication : public Application {
69 public:
70     unique_ptr<Document> createDocument() override {
71         return make_unique<SpreadsheetDocument>();
72     }
73 };
74
75 int main() {
76     unique_ptr<Application> apps[] = {
77         make_unique<WordApplication>(),
78         make_unique<PDFApplication>(),
79         make_unique<SpreadsheetApplication>()
80     };
81     for (auto& app : apps) {
82         app->newDocument();
83         cout << " --- " << endl;
84     }
85     return 0;
86 }
```

Listing 6.1: Factory Method – Document Editor

Key Insight

Factory Method uses inheritance to decide what to create. Compare with **Abstract Factory** (uses object composition and creates families of objects). Use Factory

Method when: you don't know ahead of time what class you need to instantiate, or you want subclasses to specify the objects they create.

Practice Questions – Factory Method

Easy

1. What problem does the Factory Method pattern solve?
2. Identify the roles: Product, Creator, ConcreteProduct, ConcreteCreator in a logging system.

Medium

3. Implement a Factory Method for a transport system with: Road (creates Truck), Sea (creates Ship), Air (creates Airplane). All deliver cargo.
4. How does Factory Method follow the Open/Closed Principle?

Hard

5. Extend the document editor example to support a `PresentationApplication` creating `PresentationDocument`. Show that the existing code does NOT need to change.
6. Compare Factory Method with Abstract Factory. When would you choose one over the other?

6.2 Pattern 2: Singleton (Creational)

Definition: Singleton Pattern

Intent: Ensure a class has **only one instance** and provide a global point of access to it.

Use when: Exactly one object is needed to coordinate actions – e.g., configuration manager, logger, thread pool, database connection pool.

```
1 #include <iostream>
2 #include <string>
3 #include <mutex>
4 #include <fstream>
5 using namespace std;
6
7 class Logger {
8 private:
9     // Private constructor -- prevents direct instantiation
10    Logger() {
11        logFile.open("app.log", ios::app);
12        log("Logger initialised");
13    }
```

```
14     ofstream logFile;
15     static mutex mtx; // For thread safety
16
17     // Prevent copy and assignment
18     Logger(const Logger&) = delete;
19     Logger& operator=(const Logger&) = delete;
20
21
22 public:
23     // Static method -- global access point
24     static Logger& getInstance() {
25         // C++11 guarantees static local variables are
26         initialised once
27         // and are thread-safe (magic statics)
28         static Logger instance;
29         return instance;
30     }
31
32     void log(const string& message) {
33         lock_guard<mutex> lock(mtx);
34         cout << "[LOG] " << message << endl;
35         if (logFile.is_open())
36             logFile << "[LOG] " << message << "\n";
37     }
38
39     void warn(const string& message) {
40         lock_guard<mutex> lock(mtx);
41         cout << "[WARN] " << message << endl;
42     }
43
44     void error(const string& message) {
45         lock_guard<mutex> lock(mtx);
46         cout << "[ERROR] " << message << endl;
47     }
48
49     ~Logger() {
50         if (logFile.is_open()) logFile.close();
51     }
52 };
53
54 mutex Logger::mtx;
55
56 // Configuration Singleton Example
57 class AppConfig {
58 private:
59     AppConfig() {
60         // Load from config file
61         dbHost = "localhost";
62         dbPort = 5432;
63         maxConnections = 10;
64     }
65 }
```



```
64
65     AppConfig(const AppConfig&) = delete;
66     AppConfig& operator=(const AppConfig&) = delete;
67
68     string dbHost;
69     int dbPort;
70     int maxConnections;
71
72 public:
73     static AppConfig& getInstance() {
74         static AppConfig cfg;
75         return cfg;
76     }
77
78     string getDBHost() const { return dbHost; }
79     int getDBPort() const { return dbPort; }
80     int getMaxConnections() const { return maxConnections; }
81 };
82
83 int main() {
84     Logger& log1 = Logger::getInstance();
85     Logger& log2 = Logger::getInstance();
86
87     // Both are the SAME instance
88     cout << "Same instance? " << (&log1 == &log2 ? "YES" : "NO")
89         << endl;
90
91     log1.log("Application started");
92     log2.warn("Low memory warning");
93     Logger::getInstance().error("Database connection failed");
94
95     AppConfig& cfg = AppConfig::getInstance();
96     cout << "DB Host: " << cfg.getDBHost() << ":" << cfg.
97         getDBPort() << endl;
98     return 0;
99 }
```

Listing 6.2: Singleton – Thread-safe Logger (C++11)

Common Mistake

Singletons are often considered an anti-pattern in modern software because:

- They create hidden global state.
- They make unit testing difficult (hard to mock/replace).
- They violate SRP by controlling their own creation.

Prefer **Dependency Injection** where possible. Use Singleton sparingly – only for truly single-instance resources.

Practice Questions – Singleton**Easy**

1. Why is the Singleton constructor private?
2. Name two real-world scenarios where Singleton is appropriate.

Medium

3. Implement a Singleton `ConnectionPool` that manages a fixed pool of 5 database connections.
4. Explain why Singleton can be problematic in multi-threaded applications and how C++11 solves this.

Hard

5. The Singleton is often called an “anti-pattern”. Argue both for and against this position with concrete examples.
6. Implement a `Registry` Singleton that stores key-value configuration pairs, supports thread-safe reads/writes, and can be reset for unit testing.

6.3 Pattern 3: Adapter (Structural)

Definition: Adapter Pattern

Intent: Convert the interface of a class into another interface that clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.

Also known as: Wrapper.

Two forms:

- **Object Adapter:** Uses composition (holds a reference to adaptee). Preferred.
- **Class Adapter:** Uses multiple inheritance.

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // Target Interface (what the client expects)
6 class IPaymentProcessor {
7 public:
8     virtual bool processPayment(double amount, const string&
        currency) = 0;
9     virtual bool refundPayment(double amount, const string&
        txnId) = 0;
```

```
10     virtual ~IPaymentProcessor() = default;
11 };
12
13 // Adaptee 1: Old/legacy PayPal SDK
14 class LegacyPayPal {
15 public:
16     // Different interface than what client expects
17     void makePayment(float usdAmount) {
18         cout << "PayPal: Charging $" << usdAmount << endl;
19     }
20     void reverseCharge(float usdAmount, int legacyTxnCode) {
21         cout << "PayPal: Refunding $" << usdAmount << " [code:"
22             << legacyTxnCode << "]" << endl;
23     }
24 };
25
26 // Adaptee 2: Third-party Stripe API
27 class StripeAPI {
28 public:
29     void charge(int amountCents, string currencyCode) {
30         cout << "Stripe: Charging " << amountCents << " cents ("
31             << currencyCode << ")" << endl;
32     }
33     void createRefund(string paymentIntentId) {
34         cout << "Stripe: Refunding " << paymentIntentId << endl;
35     }
36 };
37
38 // Adapter 1: PayPal Adapter (Object Adapter using composition)
39 class PayPalAdapter : public IPaymentProcessor {
40     LegacyPayPal* paypal; // Holds reference to adaptee
41 public:
42     PayPalAdapter(LegacyPayPal* pp) : paypal(pp) {}
43     bool processPayment(double amount, const string& currency)
44         override {
45         if (currency != "USD") {
46             cout << "PayPal adapter: Only USD supported" << endl;
47             ;
48             return false;
49         }
50         paypal->makePayment(static_cast<float>(amount));
51         return true;
52     }
53     bool refundPayment(double amount, const string& txnId)
54         override {
55         paypal->reverseCharge(static_cast<float>(amount), stoi(
56             txnId));
57         return true;
58     }
59 };
60
61 // Adapter 2: Stripe Adapter (Object Adapter using composition)
62 class StripeAdapter : public IPaymentProcessor {
63     StripeAPI* stripe; // Holds reference to adaptee
64 public:
65     StripeAdapter(StripeAPI* st) : stripe(st) {}
66     bool processPayment(double amount, const string& currency)
67         override {
68         if (currency != "USD") {
69             cout << "Stripe adapter: Only USD supported" << endl;
70             ;
71             return false;
72         }
73         stripe->charge(static_cast<int>(amount * 100), currency);
74         return true;
75     }
76     bool refundPayment(double amount, const string& txnId)
77         override {
78         stripe->createRefund(txnId);
79         return true;
80     }
81 };
82
83 // Client
84 class Client {
85 public:
86     void usePayPal() {
87         IPaymentProcessor* pp = new PayPalAdapter(new LegacyPayPal());
88         pp->makePayment(10.0f);
89         pp->reverseCharge(10.0f, 12345);
90         delete pp;
91     }
92     void useStripe() {
93         IPaymentProcessor* st = new StripeAdapter(new StripeAPI());
94         st->makePayment(10.0f, "USD");
95         st->refundPayment(10.0f, "pi_1234567890");
96         delete st;
97     }
98 };
99
100 int main() {
101     Client c;
102     c.usePayPal();
103     c.useStripe();
104     return 0;
105 }
```

```
55 // Adapter 2: Stripe Adapter
56 class StripeAdapter : public IPaymentProcessor {
57     StripeAPI* stripe;
58 public:
59     StripeAdapter(StripeAPI* s) : stripe(s) {}
60     bool processPayment(double amount, const string& currency)
61         override {
62         int cents = static_cast<int>(amount * 100);
63         stripe->charge(cents, currency);
64         return true;
65     }
66     bool refundPayment(double amount, const string& txnId)
67         override {
68         stripe->createRefund(txnId);
69         return true;
70     }
71 };
72
73 // Client: uses only IPaymentProcessor, unaware of specifics
74 class Checkout {
75     IPaymentProcessor* processor;
76 public:
77     Checkout(IPaymentProcessor* p) : processor(p) {}
78     void completePurchase(double amount, const string& currency,
79         const string& txnId) {
80         cout << "\nProcessing checkout..." << endl;
81         if (processor->processPayment(amount, currency))
82             cout << "Payment successful!" << endl;
83         else
84             cout << "Payment failed!" << endl;
85     }
86 };
87
88 int main() {
89     LegacyPayPal pp;
90     StripeAPI stripe;
91
92     PayPalAdapter ppAdapter(&pp);
93     StripeAdapter stripeAdapter(&stripe);
94
95     Checkout c1(&ppAdapter);
96     c1.completePurchase(99.99, "USD", "1001");
97
98     Checkout c2(&stripeAdapter);
99     c2.completePurchase(150.00, "PKR", "pi_abc123");
100    return 0;
101 }
```

Listing 6.3: Object Adapter – Payment Integration

Key Insight

Adapter vs Facade: Adapter makes an existing interface work differently; Facade provides a simpler interface to a complex subsystem. **Adapter vs Decorator:** Adapter changes the interface; Decorator adds behaviour while keeping the interface the same.

Practice Questions – Adapter**Easy**

1. What is the purpose of the Adapter pattern? Give a real-world analogy.
2. What is the difference between Object Adapter and Class Adapter?

Medium

3. Implement an Adapter for a `LegacyStack` (with methods `push_item`, `pop_item`) to conform to a modern `IStack` interface (`push`, `pop`, `peek`).
4. How does the Adapter pattern relate to the Open/Closed Principle?

Hard

5. Design an adapter system for integrating three different weather APIs (OpenWeather, WeatherAPI, AccuWeather) each with different response formats into a unified `IWeatherService` interface.

6.4 Pattern 4: Composite (Structural)

Definition: Composite Pattern

Intent: Compose objects into **tree structures** to represent part-whole hierarchies. Composite lets clients treat individual objects (Leaf) and compositions of objects (Composite) **uniformly**.

Key idea: Both a single leaf and a container holding many leaves respond to the same interface.

```
1 #include <iostream>
2 #include <vector>
3 #include <memory>
4 #include <string>
5 #include <iomanip>
6 using namespace std;
7
8 // Component (Abstract)
9 class FileSystemItem {
10 protected:
11     string name;
12 public:
```

```

13     FileSystemItem(const string& n) : name(n) {}
14     virtual void display(int depth = 0) const = 0;
15     virtual long long getSize() const = 0;
16     virtual bool isDirectory() const = 0;
17     const string& getName() const { return name; }
18     virtual ~FileSystemItem() = default;
19 };
20
21 // Leaf: File
22 class File : public FileSystemItem {
23     long long size;
24     string extension;
25 public:
26     File(const string& n, long long sz, const string& ext = "")
27         : FileSystemItem(n), size(sz), extension(ext) {}
28
29     void display(int depth = 0) const override {
30         cout << string(depth * 2, ' ') << " [FILE] "
31             << name << "." << extension
32             << " (" << size << " bytes)" << endl;
33     }
34     long long getSize() const override { return size; }
35     bool isDirectory() const override { return false; }
36 };
37
38 // Composite: Directory
39 class Directory : public FileSystemItem {
40     vector<shared_ptr<FileSystemItem>> children;
41 public:
42     Directory(const string& n) : FileSystemItem(n) {}
43
44     void add(shared_ptr<FileSystemItem> item) {
45         children.push_back(item);
46     }
47
48     void remove(const string& itemName) {
49         children.erase(
50             remove_if(children.begin(), children.end(),
51                 [&](const shared_ptr<FileSystemItem>& item) {
52                     return item->getName() == itemName;
53                 }),
54             children.end()
55         );
56     }
57
58     // Recursive display -- works for any depth
59     void display(int depth = 0) const override {
60         cout << string(depth * 2, ' ') << "+ [DIR] " << name <<
61             "/" << endl;
62         for (const auto& child : children)
63             child->display(depth + 1);

```

```

63     }
64
65     // Recursive size -- sums all nested items
66     long long getSize() const override {
67         long long total = 0;
68         for (const auto& child : children)
69             total += child->getSize();
70         return total;
71     }
72
73     bool isDirectory() const override { return true; }
74
75     int itemCount() const {
76         int count = 0;
77         for (const auto& c : children) {
78             count++;
79             if (c->isDirectory()) {
80                 // Count recursively inside dirs too
81             }
82         }
83         return count;
84     }
85 };
86
87 int main() {
88     // Build file system tree
89     auto root = make_shared<Directory>("C:");
90     auto docs = make_shared<Directory>("Documents");
91     auto pics = make_shared<Directory>("Pictures");
92     auto work = make_shared<Directory>("Work");
93
94     docs->add(make_shared<File>("resume", 50000, "pdf"));
95     docs->add(make_shared<File>("notes", 3200, "txt"));
96     work->add(make_shared<File>("report", 125000, "docx"));
97     work->add(make_shared<File>("budget", 45000, "xlsx"));
98     docs->add(work);
99
100    pics->add(make_shared<File>("holiday", 4200000, "jpg"));
101    pics->add(make_shared<File>("profile", 210000, "png"));
102
103    root->add(docs);
104    root->add(pics);
105    root->add(make_shared<File>("autoexec", 512, "bat"));
106
107    // Uniform treatment: display works the same for file or
108    directory
109    cout << "=== File System ===" << endl;
110    root->display();
111
112    cout << "\nTotal size: " << root->getSize() << " bytes" <<
        endl;

```

```
112     cout << "Docs size: " << docs->getSize() << " bytes" <<
        endl;
113
114     // A single file also responds to the same interface
115     File singleFile("test", 1024, "cpp");
116     singleFile.display();
117     cout << "File size: " << singleFile.getSize() << " bytes" <<
        endl;
118
119     return 0;
120 }
```

Listing 6.4: Composite Pattern – File System

Practice Questions – Composite

Easy

1. What is the key idea of the Composite pattern? What does “uniform treatment” mean?
2. Identify Leaf, Composite, and Component in the file system example.

Medium

3. Implement a Composite pattern for a **Menu** system where a menu can contain menu items (leaves) or sub-menus (composites). Display the full menu tree with indentation.
4. What design principle does Composite most directly support?

Hard

5. Design a Composite structure for an organisational hierarchy (CEO → Managers → Employees). Implement methods to: calculate total salary of a division, count total employees under a manager, and find the highest-paid employee recursively.

6.5 Pattern 5: Iterator (Behavioral)

Definition: Iterator Pattern

Intent: Provide a way to **sequentially access elements** of a collection without exposing the underlying representation.

Key benefit: The client traversal code is decoupled from the collection’s implementation. Same client code works with arrays, lists, trees, etc.

```
1 #include <iostream>
2 #include <vector>
3 #include <stdexcept>
```



```
4 #include <string>
5 using namespace std;
6
7 // Iterator Interface
8 template<typename T>
9 class Iterator {
10 public:
11     virtual bool hasNext() const = 0;
12     virtual T next() = 0;
13     virtual void reset() = 0;
14     virtual ~Iterator() = default;
15 };
16
17 // Collection Interface
18 template<typename T>
19 class Collection {
20 public:
21     virtual Iterator<T>* createIterator() const = 0;
22     virtual ~Collection() = default;
23 };
24
25 // Concrete Collection: StudentList
26 class StudentList : public Collection<string> {
27     vector<string> students;
28 public:
29     void addStudent(const string& name) { students.push_back(
30         name); }
31     size_t size() const { return students.size(); }
32     const string& get(size_t index) const { return students.at(
33         index); }
34
35     // Concrete Iterator (inner-class style via separate class)
36     class StudentIterator : public Iterator<string> {
37         const StudentList& list;
38         size_t current;
39     public:
40         StudentIterator(const StudentList& l) : list(l), current
41             (0) {}
42         bool hasNext() const override { return current < list.
43             size(); }
44         string next() override {
45             if (!hasNext()) throw runtime_error("No more
46                 elements");
47             return list.get(current++);
48         }
49         void reset() override { current = 0; }
50     };
51
52     Iterator<string>* createIterator() const override {
53         return new StudentIterator(*this);
54     }
55 }
```

```

50 };
51
52 // Concrete Collection: FilteredStudentList (only names
   starting with 'A')
53 class FilteredStudentIterator : public Iterator<string> {
54     const StudentList& list;
55     size_t current;
56     char filterChar;
57     void advanceToNext() {
58         while (current < list.size() && list.get(current)[0] !=
               filterChar)
59             current++;
60     }
61 public:
62     FilteredStudentIterator(const StudentList& l, char fc)
63         : list(l), current(0), filterChar(fc) { advanceToNext();
        }
64     bool hasNext() const override { return current < list.size()
        ; }
65     string next() override {
66         if (!hasNext()) throw runtime_error("No more elements");
67         string result = list.get(current++);
68         advanceToNext();
69         return result;
70     }
71     void reset() override { current = 0; advanceToNext(); }
72 };
73
74 int main() {
75     StudentList students;
76     students.addStudent("Ali");
77     students.addStudent("Sara");
78     students.addStudent("Ahmed");
79     students.addStudent("Aisha");
80     students.addStudent("Bilal");
81     students.addStudent("Amna");
82
83     // Standard iteration
84     cout << "=== All Students ===" << endl;
85     Iterator<string>* it = students.createIterator();
86     while (it->hasNext())
87         cout << " " << it->next() << endl;
88     delete it;
89
90     // Filtered iteration -- only 'A' names
91     cout << "\n=== Students starting with 'A' ===" << endl;
92     FilteredStudentIterator filtered(students, 'A');
93     while (filtered.hasNext())
94         cout << " " << filtered.next() << endl;
95
96     return 0;

```

97 }

Listing 6.5: Iterator Pattern – Custom Collection

Key Insight

C++ STL heavily uses the Iterator pattern: `std::vector::iterator`, `std::list::iterator`, range-based for loops (`for(auto x : collection)`). The Iterator pattern decouples the algorithm from the data structure.

Practice Questions – Iterator**Easy**

1. What is the main benefit of the Iterator pattern?
2. What methods does a typical Iterator interface define?

Medium

3. Implement an Iterator for a circular linked list that traverses elements in a round-robin fashion.
4. How is the Iterator pattern used in the C++ STL? Give two concrete examples.

Hard

5. Implement a `BinaryTreeIterator` that traverses a binary search tree in **in-order** sequence using the Iterator pattern. The client should be able to iterate with `hasNext()` and `next()` without knowing the tree's internals.

6.6 Pattern 6: Observer (Behavioral)

Definition: Observer Pattern

Intent: Define a one-to-many dependency between objects so that when one object (Subject) changes state, all its dependents (Observers) are notified and updated automatically.

Also known as: Publish-Subscribe, Event-Listener.

Use when: A change in one object requires changing others, and you don't know how many objects need to change.

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <algorithm>
5 #include <memory>
6 using namespace std;
7
```

```
8 // Observer Interface
9 class StockObserver {
10 public:
11     virtual void update(const string& stockSymbol,
12                         double oldPrice, double newPrice) = 0;
13     virtual string getName() const = 0;
14     virtual ~StockObserver() = default;
15 };
16
17 // Subject Interface
18 class StockSubject {
19 public:
20     virtual void subscribe(StockObserver* obs) = 0;
21     virtual void unsubscribe(StockObserver* obs) = 0;
22     virtual void notifyObservers() = 0;
23     virtual ~StockSubject() = default;
24 };
25
26 // Concrete Subject: Stock
27 class Stock : public StockSubject {
28     string symbol;
29     double price;
30     vector<StockObserver*> observers;
31 public:
32     Stock(const string& sym, double initialPrice)
33         : symbol(sym), price(initialPrice) {}
34
35     void subscribe(StockObserver* obs) override {
36         observers.push_back(obs);
37         cout << obs->getName() << " subscribed to " << symbol <<
38             endl;
39     }
40
41     void unsubscribe(StockObserver* obs) override {
42         observers.erase(remove(observers.begin(), observers.end
43             (), obs),
44             observers.end());
45         cout << obs->getName() << " unsubscribed from " <<
46             symbol << endl;
47     }
48
49     void notifyObservers() override {
50         for (auto obs : observers)
51             obs->update(symbol, price, price); // will be called
52             with new
53     }
54
55     // Change price and notify all observers
56     void setPrice(double newPrice) {
57         double old = price;
58         price = newPrice;
```

```

55         cout << "\n[STOCK] " << symbol << " price: " << old
56             << " -> " << newPrice << endl;
57         for (auto obs : observers)
58             obs->update(symbol, old, newPrice);
59     }
60
61     double getPrice() const { return price; }
62     const string& getSymbol() const { return symbol; }
63 };
64
65 // Concrete Observers
66 class MobileApp : public StockObserver {
67     string appName;
68     double alertThreshold;
69 public:
70     MobileApp(const string& name, double threshold = 5.0)
71         : appName(name), alertThreshold(threshold) {}
72
73     void update(const string& sym, double oldP, double newP)
74         override {
75         double change = ((newP - oldP) / oldP) * 100.0;
76         cout << " [" << appName << "]" << sym
77             << " changed by " << fixed << setprecision(2) <<
78             change << "%" << endl;
79         if (abs(change) >= alertThreshold)
80             cout << " [" << appName << "]" << "ALERT! Significant
81             change!" << endl;
82     }
83     string getName() const override { return appName; }
84 };
85
86 class TradingBot : public StockObserver {
87     string botName;
88     double buyBelow, sellAbove;
89 public:
90     TradingBot(const string& name, double buy, double sell)
91         : botName(name), buyBelow(buy), sellAbove(sell) {}
92
93     void update(const string& sym, double oldP, double newP)
94         override {
95         if (newP < buyBelow)
96             cout << " [" << botName << "]" << "BUY signal for " <<
97             sym
98             << " at " << newP << endl;
99         else if (newP > sellAbove)
100             cout << " [" << botName << "]" << "SELL signal for " <<
101             sym
102             << " at " << newP << endl;
103     }
104     string getName() const override { return botName; }
105 };

```

```

100
101 class PortfolioTracker : public StockObserver {
102     string trackerName;
103     double totalGainLoss = 0.0;
104 public:
105     PortfolioTracker(const string& name) : trackerName(name) {}
106     void update(const string& sym, double oldP, double newP)
107         override {
108         totalGainLoss += (newP - oldP);
109         cout << " [" << trackerName << "] Portfolio P&L: "
110             << totalGainLoss << endl;
111     }
112     string getName() const override { return trackerName; }
113 };
114
115 int main() {
116     Stock appl("APPL", 150.0);
117     Stock googl("GOOGL", 2800.0);
118
119     MobileApp      mobileUser("StockApp", 3.0);
120     TradingBot     bot("AutoTrader", 140.0, 165.0);
121     PortfolioTracker tracker("MyPortfolio");
122
123     appl.subscribe(&mobileUser);
124     appl.subscribe(&bot);
125     appl.subscribe(&tracker);
126     googl.subscribe(&mobileUser);
127
128     appl.setPrice(158.0);
129     appl.setPrice(135.0);
130
131     appl.unsubscribe(&bot);
132     appl.setPrice(170.0); // Bot no longer notified
133
134     googl.setPrice(2950.0);
135     return 0;
136 }

```

Listing 6.6: Observer Pattern – Stock Market Ticker

Practice Questions – Observer

Easy

1. What is the “one-to-many dependency” in Observer? Give a real-world example.
2. What are Subject and Observer? What methods does each have?

Medium

3. Implement an Observer for a weather station (Subject: WeatherStation; Ob-

servers: PhoneDisplay, WebDisplay, StatisticsDisplay) that shows temperature/humidity changes.

4. How does Observer relate to the Open/Closed Principle?

Hard

5. Implement a push vs pull variant of Observer. In the *push* model, the Subject sends all data in `update()`. In the *pull* model, the Observer holds a reference to the Subject and fetches only what it needs. Compare the trade-offs.
6. Implement an event system where Observers can subscribe to **specific event types** (e.g., “price_up”, “price_down”, “volume_spike”) rather than all events from a subject.

6.7 Pattern 7: State (Behavioral)

Definition: State Pattern

Intent: Allow an object to alter its **behaviour when its internal state changes**. The object will appear to change its class.

Key idea: Instead of large if/switch statements checking state, encapsulate state-specific behaviour in separate State classes. The Context delegates to the current State object.

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class VendingMachine; // Forward declaration
6
7 // State Interface
8 class VendingState {
9 public:
10     virtual void insertCoin(VendingMachine& vm) = 0;
11     virtual void selectProduct(VendingMachine& vm) = 0;
12     virtual void dispense(VendingMachine& vm) = 0;
13     virtual void cancel(VendingMachine& vm) = 0;
14     virtual string getStateName() const = 0;
15     virtual ~VendingState() = default;
16 };
17
18 // Context
19 class VendingMachine {
20     VendingState* currentState;
21     int itemCount;
22     double coinInserted;
23     string selectedProduct;
24 public:

```

```

25     VendingMachine(int items);
26
27     void setState(VendingState* newState) { currentState =
        newState; }
28     VendingState* getState() const { return currentState; }
29
30     int getItemCount() const { return itemCount; }
31     void setItemCount(int c) { itemCount = c; }
32
33     double getCoinInserted() const { return coinInserted; }
34     void setCoinInserted(double c) { coinInserted = c; }
35
36     const string& getSelectedProduct() const { return
        selectedProduct; }
37     void setSelectedProduct(const string& p) { selectedProduct =
        p; }
38
39     // Delegate to current state
40     void insertCoin() { currentState->insertCoin(*
        this); }
41     void selectProduct(const string& p) {
42         setSelectedProduct(p);
43         currentState->selectProduct(*this);
44     }
45     void dispense() { currentState->dispense(*this); }
46     void cancel() { currentState->cancel(*this); }
47
48     void showStatus() const {
49         cout << "[Machine] State: " << currentState->
            getStateName()
50             << " | Items: " << itemCount
51             << " | Coin: Rs." << coinInserted << endl;
52     }
53 };
54
55 // Concrete States
56 class IdleState : public VendingState {
57 public:
58     void insertCoin(VendingMachine& vm) override;
59     void selectProduct(VendingMachine& vm) override {
60         cout << "[Idle] Insert coin first!" << endl;
61     }
62     void dispense(VendingMachine& vm) override {
63         cout << "[Idle] No transaction in progress" << endl;
64     }
65     void cancel(VendingMachine& vm) override {
66         cout << "[Idle] Nothing to cancel" << endl;
67     }
68     string getStateName() const override { return "Idle"; }
69 };
70

```



```

71 class HasCoinState : public VendingState {
72 public:
73     void insertCoin(VendingMachine& vm) override {
74         cout << "[HasCoin] Coin already inserted. Select product
75             ." << endl;
76     }
77     void selectProduct(VendingMachine& vm) override;
78     void dispense(VendingMachine& vm) override {
79         cout << "[HasCoin] Select a product first!" << endl;
80     }
81     void cancel(VendingMachine& vm) override;
82     string getStateName() const override { return "HasCoin"; }
83 };
84
85 class ProductSelectedState : public VendingState {
86 public:
87     void insertCoin(VendingMachine& vm) override {
88         cout << "[Selected] Already dispensing..." << endl;
89     }
90     void selectProduct(VendingMachine& vm) override {
91         cout << "[Selected] Already selected" << endl;
92     }
93     void dispense(VendingMachine& vm) override;
94     void cancel(VendingMachine& vm) override;
95     string getStateName() const override { return "
96         ProductSelected"; }
97 };
98
99 class SoldOutState : public VendingState {
100 public:
101     void insertCoin(VendingMachine& vm) override {
102         cout << "[SoldOut] Machine is empty. Returning coin." <<
103             endl;
104     }
105     void selectProduct(VendingMachine& vm) override {
106         cout << "[SoldOut] Machine is empty." << endl;
107     }
108     void dispense(VendingMachine& vm) override {
109         cout << "[SoldOut] No items." << endl;
110     }
111     void cancel(VendingMachine& vm) override {
112         cout << "[SoldOut] Nothing to cancel." << endl;
113     }
114     string getStateName() const override { return "SoldOut"; }
115 };
116
117 // Global state instances (could use shared_ptr)
118 static IdleState idleState;
119 static HasCoinState hasCoinState;
120 static ProductSelectedState selectedState;
121 static SoldOutState soldOutState;

```

```

119
120 // VendingMachine constructor
121 VendingMachine::VendingMachine(int items)
122     : itemCount(items), coinInserted(0.0) {
123     currentState = (items > 0) ? (VendingState*)&idleState
124                               : (VendingState*)&soldOutState;
125 }
126
127 // State method implementations
128 void IdleState::insertCoin(VendingMachine& vm) {
129     vm.setCoinInserted(50.0); // Assume Rs. 50 coin
130     cout << "[Idle] Coin inserted (Rs.50). Now select a product."
131           << endl;
132     vm.setState(&hasCoinState);
133 }
134 void HasCoinState::selectProduct(VendingMachine& vm) {
135     cout << "[HasCoin] Product '" << vm.getSelectedProduct()
136           << "' selected. Dispensing..." << endl;
137     vm.setState(&selectedState);
138     vm.dispense();
139 }
140
141 void HasCoinState::cancel(VendingMachine& vm) {
142     cout << "[HasCoin] Transaction cancelled. Returning Rs."
143           << vm.getCoinInserted() << endl;
144     vm.setCoinInserted(0.0);
145     vm.setState(&idleState);
146 }
147
148 void ProductSelectedState::dispense(VendingMachine& vm) {
149     cout << "[Dispensing] '" << vm.getSelectedProduct() << "'
150           << "dispensed!" << endl;
151     vm.setItemCount(vm.getItemCount() - 1);
152     vm.setCoinInserted(0.0);
153     if (vm.getItemCount() == 0) {
154         cout << "[Machine] SOLD OUT!" << endl;
155         vm.setState(&soldOutState);
156     } else {
157         vm.setState(&idleState);
158     }
159 }
160 void ProductSelectedState::cancel(VendingMachine& vm) {
161     cout << "[Selected] Cannot cancel during dispensing" << endl;
162     ;
163 }
164
165 int main() {
166     VendingMachine machine(3);
167     machine.showStatus();

```

```
167     machine.insertCoin();
168     machine.selectProduct("Pepsi");
169     machine.showStatus();
170
171     machine.insertCoin();
172     machine.cancel();
173     machine.showStatus();
174
175     machine.insertCoin();
176     machine.selectProduct("Lays");
177     machine.showStatus();
178
179     machine.insertCoin();
180     machine.selectProduct("Water");
181     machine.showStatus();
182
183     // Machine now sold out
184     machine.insertCoin();
185
186     return 0;
187 }
188
```

Listing 6.7: State Pattern – Vending Machine

Practice Questions – State

Easy

1. What problem does the State pattern solve? Why is a large **switch** statement a bad alternative?
2. Identify Context, State interface, and ConcreteStates in the vending machine example.

Medium

3. Implement a State pattern for a **TrafficLight** with states: Red (30s), Yellow (5s), Green (25s). Transitions cycle automatically.
4. How does the State pattern differ from Strategy? (Both use a context that delegates to a strategy/state object.)

Hard

5. Design a State pattern for an **ATM** with states: Idle, CardInserted, PINEntered, TransactionInProgress, OutOfCash. Include all transitions with guards (e.g., wrong PIN ≤ 3 attempts).

6.8 Pattern 8: Template Method (Behavioral)

Definition: Template Method Pattern

Intent: Define the **skeleton of an algorithm** in a base class, deferring some steps to subclasses. Template Method lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.

Key idea: The invariant parts of the algorithm stay in the base class. The variant parts are abstract (or overrideable) methods called **primitive operations**.

```
1 #include <iostream>
2 #include <string>
3 #include <vector>
4 using namespace std;
5
6 // Abstract Class with Template Method
7 class DataMiner {
8 public:
9     // Template Method: defines the algorithm skeleton
10    // Declared final so subclasses cannot change the order of
11    // steps
12    void mine(const string& path) {
13        openFile(path);
14        extractData();
15        parseData();
16        analyzeData();
17        sendReport();
18        closeFile();
19    }
20 protected:
21    // Abstract primitive operations (must override)
22    virtual void openFile(const string& path) = 0;
23    virtual void extractData() = 0;
24    virtual void parseData() = 0;
25
26    // Hook: optional override (has default implementation)
27    virtual void analyzeData() {
28        cout << " [Default] Basic statistical analysis..." <<
29        endl;
30    }
31
32    // Concrete step: same for all subclasses
33    void sendReport() {
34        cout << " Sending report via email..." << endl;
35    }
36    void closeFile() {
37        cout << " Closing file." << endl;
38    }
```

```
39 public:
40     virtual ~DataMiner() = default;
41 };
42
43 // Concrete Miner 1: PDF Data Miner
44 class PDFDataMiner : public DataMiner {
45     vector<string> rawData;
46 protected:
47     void openFile(const string& path) override {
48         cout << "[PDF Miner] Opening PDF: " << path << endl;
49     }
50     void extractData() override {
51         cout << "[PDF Miner] Extracting text from PDF pages..."
52             << endl;
53         rawData = {"Page1 data", "Page2 data", "Page3 data"};
54     }
55     void parseData() override {
56         cout << "[PDF Miner] Parsing text with NLP..." << endl;
57         for (auto& d : rawData)
58             cout << "    Parsed: " << d << endl;
59     }
60     void analyzeData() override {
61         cout << "[PDF Miner] Running sentiment analysis on
62             parsed text..." << endl;
63     }
64 };
65
66 // Concrete Miner 2: CSV Data Miner
67 class CSVDataMiner : public DataMiner {
68     vector<vector<string>> table;
69 protected:
70     void openFile(const string& path) override {
71         cout << "[CSV Miner] Opening CSV: " << path << endl;
72     }
73     void extractData() override {
74         cout << "[CSV Miner] Reading rows from CSV..." << endl;
75         table = {{ "Ali", "85" }, { "Sara", "92" }, { "Ahmed", "78" }};
76     }
77     void parseData() override {
78         cout << "[CSV Miner] Parsing columns and data types..."
79             << endl;
80         for (auto& row : table)
81             cout << "    " << row[0] << ": " << row[1] << endl;
82     }
83     // Uses default analyzeData() -- no override needed
84 };
85
86 // Concrete Miner 3: Database Miner
87 class DatabaseMiner : public DataMiner {
88 protected:
89     void openFile(const string& path) override {
```

```

87         cout << "[DB Miner] Connecting to DB: " << path << endl;
88     }
89     void extractData() override {
90         cout << "[DB Miner] Running SQL query: SELECT * FROM
          sales..." << endl;
91     }
92     void parseData() override {
93         cout << "[DB Miner] Mapping ResultSet to objects..." <<
          endl;
94     }
95     void analyzeData() override {
96         cout << "[DB Miner] Running OLAP cube analysis..." <<
          endl;
97     }
98 };
99
100 // Data Exporter with Template Method
101 class DataExporter {
102 public:
103     // Template method
104     void exportData(const vector<string>& data) {
105         prepareHeader();
106         for (const auto& item : data)
107             exportItem(item);
108         writeFooter();
109     }
110 protected:
111     virtual void prepareHeader() = 0;
112     virtual void exportItem(const string& item) = 0;
113     virtual void writeFooter() {} // Hook: optional
114 };
115
116 class JSONExporter : public DataExporter {
117 protected:
118     void prepareHeader() override { cout << "[\n"; }
119     void exportItem(const string& item) override {
120         cout << "    {\"value\": \"" << item << "\"},\n";
121     }
122     void writeFooter() override { cout << "]\n"; }
123 };
124
125 class XMLExporter : public DataExporter {
126 protected:
127     void prepareHeader() override { cout << "<data>\n"; }
128     void exportItem(const string& item) override {
129         cout << "    <item>" << item << "</item>\n";
130     }
131     void writeFooter() override { cout << "</data>\n"; }
132 };
133
134 int main() {

```

```
135     cout << "=== PDF Mining ===" << endl;
136     PDFDataMiner pdfMiner;
137     pdfMiner.mine("annual_report.pdf");
138
139     cout << "\n=== CSV Mining ===" << endl;
140     CSVDataMiner csvMiner;
141     csvMiner.mine("grades.csv");
142
143     cout << "\n=== Database Mining ===" << endl;
144     DatabaseMiner dbMiner;
145     dbMiner.mine("jdbc:mysql://localhost/sales");
146
147     cout << "\n=== Data Export ===" << endl;
148     vector<string> data = {"Item1", "Item2", "Item3"};
149     JSONExporter jsonExp;
150     jsonExp.exportData(data);
151     XMLExporter xmlExp;
152     xmlExp.exportData(data);
153
154     return 0;
155 }
```

Listing 6.8: Template Method – Data Mining Reports

Key Insight

Template Method vs Strategy:

- Template Method: Uses **inheritance** to vary parts of an algorithm. Algorithm structure is in the base class.
- Strategy: Uses **composition/delegation** to swap the entire algorithm at runtime.

Template Method follows the **Hollywood Principle**: “Don’t call us, we’ll call you” – the base class calls the subclass methods, not vice versa.

Practice Questions – Template Method

Easy

1. What is a “primitive operation” in Template Method? What is a “hook”?
2. Why is the template method sometimes declared **final** in C++?

Medium

3. Implement a Template Method for a **GameAI** class with steps: **takeTurn()** = **collectResources()** + **buildStructures()** + **buildUnits()** + **attack()**. Implement two AIs: **OrcsAI** and **MonstersAI**.
4. What is the Hollywood Principle? How does Template Method embody it?

Hard

5. Implement a Template Method for a multi-step `OrderFulfillment` pipeline: validate order, reserve inventory, charge payment, generate shipment label, send confirmation. Some steps (like charge payment) must be abstract; others (like send confirmation) should have default implementations. Implement two variants: `DigitalOrderFulfillment` (no shipment label) and `PhysicalOrderFulfillment`.

Chapter 7

Additional Design Principles

7.1 DRY – Don’t Repeat Yourself

Definition: DRY

Every piece of knowledge must have a single, unambiguous, authoritative representation within a system. Duplicated code leads to inconsistency when one copy is updated but others are not.

```
1 // BAD: Validation logic duplicated
2 void registerUser(const string& email) {
3     if (email.find('@') == string::npos)
4         throw invalid_argument("Invalid email");
5     // ... register logic
6 }
7 void updateEmail(const string& email) {
8     if (email.find('@') == string::npos) // DUPLICATE!
9         throw invalid_argument("Invalid email");
10    // ... update logic
11 }
12
13 // GOOD: Single source of truth
14 bool isValidEmail(const string& email) {
15     return email.find('@') != string::npos
16         && email.find('.') != string::npos;
17 }
18 void registerUser(const string& email) {
19     if (!isValidEmail(email)) throw invalid_argument("Invalid
20     email");
21 }
22 void updateEmail(const string& email) {
23     if (!isValidEmail(email)) throw invalid_argument("Invalid
24     email");
25 }
```

Listing 7.1: DRY Violation and Fix

7.2 KISS – Keep It Simple, Stupid

Definition: KISS

Most systems work best when they are kept simple. Avoid unnecessary complexity. Prefer the simplest solution that solves the problem correctly.

7.3 YAGNI – You Ain’t Gonna Need It

Definition: YAGNI

Do not add functionality until it is actually needed. Over-engineering for hypothetical future requirements is wasteful and introduces unnecessary complexity.

7.4 Law of Demeter (LoD)

Definition: Law of Demeter (Principle of Least Knowledge)

A method M of class C should only call methods of:

1. C itself
2. Objects passed as arguments to M
3. Objects created inside M
4. Direct components of C (instance variables)

Informally: Only talk to your immediate friends, not strangers. Avoid “train wrecks”: `a.getB().getC().doSomething()`.

```
1 // BAD: Train wreck -- violates LoD
2 class Order {
3     Customer customer;
4 public:
5     void process() {
6         // Reaching through customer -> wallet -> money
7         string city = customer.getAddress().getCity().
8             toUpperCase(); // VIOLATION
9     }
10 };
11 // GOOD: Ask the direct object for what you need
12 class Customer {
13     Address address;
14 public:
15     string getCity() const { return address.getCity(); } //
16     Wrapper
```

```

16 };
17 class Order {
18     Customer customer;
19 public:
20     void process() {
21         string city = customer.getCity(); // Only talks to
           Customer
22     }
23 };

```

Listing 7.2: Law of Demeter Violation and Fix

7.5 Composition Over Inheritance

Definition: Composition Over Inheritance

Favour composing objects to achieve behaviour over creating complex inheritance hierarchies. This leads to more flexible, maintainable designs.

```

1  // BAD: Deep inheritance for different duck behaviours
2  class Duck { virtual void fly() {} virtual void quack() {} };
3  class RubberDuck : public Duck { // Can't fly, can't quack
      properly
4      void fly() override { /* do nothing */ }
5      void quack() override { cout << "Squeak!" << endl; }
6  };
7
8  // GOOD: Compose behaviours
9  class FlyBehaviour {
10 public:
11     virtual void fly() = 0;
12     virtual ~FlyBehaviour() = default;
13 };
14 class CanFly : public FlyBehaviour {
15     void fly() override { cout << "Flying with wings!" << endl; }
16 };
17 class CannotFly : public FlyBehaviour {
18     void fly() override { cout << "Cannot fly." << endl; }
19 };
20
21 class Duck {
22     unique_ptr<FlyBehaviour> flyBehaviour;
23 public:
24     Duck(unique_ptr<FlyBehaviour> fb) : flyBehaviour(move(fb))
        {}
25     void performFly() { flyBehaviour->fly(); }
26     // Behaviour can be swapped at runtime!
27     void setFlyBehaviour(unique_ptr<FlyBehaviour> fb) {
28         flyBehaviour = move(fb);

```

```
29     }  
30 };
```

Listing 7.3: Prefer Composition

Practice Questions – Design Principles

Easy

1. What does YAGNI mean? Give an example of over-engineering.
2. What is a “train wreck” in the context of LoD?

Medium

3. Identify all SOLID and other design principle violations in this class:

```
class UserManager {  
    void addUser(string name, int age, string email) {  
        /* adds to DB */ }  
    void sendWelcomeEmail(string email) { /* SMTP logic */ }  
    void validateAge(int age) { if(age < 0) throw; }  
    void generatePDFReport() { /* report logic */ }  
};
```

4. Explain “composition over inheritance” with a shape-drawing example where shapes can be filled, stroked, shadowed – all independently.

Hard

5. Design a plugin-based text processor where operations (spell-check, grammar-check, word-count, format) are plugged in at runtime using composition. Show how this is superior to an inheritance-based approach and how it satisfies OCP, SRP, and DIP.

Chapter 8

Quick Reference & Exam Tips

8.1 Design Pattern Summary Table

Pattern	Category	Intent	Key Participants
Factory Method	Creational	Defer instantiation to subclasses	Creator, ConcreteCreator, Product
Singleton	Creational	Ensure single instance	Singleton class with static <code>getInstance()</code>
Adapter	Structural	Convert interface	Target, Adaptee, Adapter, Client
Composite	Structural	Tree structures, uniform treatment	Component, Leaf, Composite
Iterator	Behavioral	Sequential access without exposing internals	Iterator, ConcreteIterator, Collection
Observer	Behavioral	Notify dependents of state changes	Subject, Observer, ConcreteObserver
State	Behavioral	Change behaviour with state	Context, State, ConcreteState
Template Method	Behavioral	Skeleton algorithm, defer steps	AbstractClass, ConcreteClass

8.2 SOLID Principles – One-liner Summary

Principle	Rule	Violation Signal
SRP	One class, one reason to change	Class does too many things
OCP	Extend, don't modify	Adding features breaks existing code
LSP	Subclass must be substitutable	Subclass throws exceptions for base behaviour
ISP	No fat interfaces	Classes implement unused methods
DIP	Depend on abstractions	<code>new ConcreteClass()</code> in high-level code

8.3 CK Metrics Quick Reference

Metric	Formula/Definition	Threshold	Problem if High
WMC	$\sum$ method complexities	≤ 20	Complex class
DIT	Longest inheritance path	2–5	Fragile hierarchy
NOC	# immediate subclasses	depends	Misuse of inheritance
CBO	# classes coupled to	≤ 14	Low modularity
RFC	$ M + R $	≤ 50	Hard to test
LCOM	$\max(P - Q , 0)$	$= 0$	Low cohesion

8.4 Common Exam Mistakes to Avoid

Common Mistake

1. **Association vs Aggregation vs Composition:** Remember – composition = strong ownership, parts die with whole; aggregation = weak ownership.
2. **LSP:** The Square/Rectangle example is the classic trap. If a subclass can't honour all base class contracts, it violates LSP.
3. **Singleton:** Always explain thread safety. C++11 static locals are thread-safe.
4. **Factory Method vs Abstract Factory:** Factory Method uses inheritance; Abstract Factory uses object composition to create families of related objects.
5. **Observer push vs pull:** In push, Subject sends all data; in pull, Observer queries Subject. Know trade-offs.
6. **Template Method vs Strategy:** Template = inheritance; Strategy = composition. Template fixes algorithm structure; Strategy swaps entire algorithm.
7. **LCOM:** A value of 0 is GOOD (cohesive); a high value is BAD.
8. **DIT:** Both extremes are bad – 0 means no reuse; > 5 means too deep.